# Web Application Penetration Test Report

Target: DVWA (Damn Vulnerable Web Application)
Security Level Tested: Low

---

| | |
|---|---|
| **Prepared by:** | Humayun |
| **Program:** | CloudExify Cybersecurity Summer Internship 2026 |
| **Registration No.:** | CX-INT-2026-CYB-0052 |
| **Project:** | Month 2 — Project 4 (Final Capstone) |
| **Report type:** | Web Application Penetration Test Report (manual testing) |
| **Primary tool:** | Burp Suite Community Edition v2026.7.3 |
| **Test date:** | August 2026 |

# 1. Executive Summary

This report documents manual penetration testing performed against DVWA (Damn Vulnerable Web Application), running on Apache/MySQL/PHP via XAMPP, using Burp Suite Community Edition as an intercepting proxy and request-replay tool (Repeater). Testing targeted the application's authentication, input-handling, and file-upload functionality with the security level set to Low.

Four vulnerabilities were manually confirmed: **SQL Injection** (Critical), **Unrestricted File Upload** (Critical), **Stored Cross-Site Scripting** (High), and **Reflected Cross-Site Scripting** (Medium). Each was reproduced and captured with request/response evidence, and each carries a business impact ranging from full application compromise (SQL Injection, File Upload) to session hijacking and credential theft against other users of the application (XSS).

**Overall risk level: CRITICAL.** The combination of unauthenticated SQL injection and unrestricted file upload means an attacker can both exfiltrate the entire user database and execute arbitrary PHP code on the server, representing complete compromise of the application and host.

## Findings Summary

| # | Vulnerability | CVSS v3.0 | Severity |
|---|---|---|---|
| 1 | SQL Injection (UNION-based) | 9.8 | **CRITICAL** |
| 2 | Unrestricted File Upload | 9.8 | **CRITICAL** |
| 3 | Stored Cross-Site Scripting (XSS) | 8.8 | **HIGH** |
| 4 | Reflected Cross-Site Scripting (XSS) | 6.1 | **MEDIUM** |

# 2. Testing Methodology & Scope

Testing was conducted from a Kali Linux 2026.2 VM (VirtualBox, bridged networking) against DVWA hosted on the Windows machine via XAMPP. Burp Suite Community Edition's browser proxy and Repeater tool were used to intercept, modify, and replay HTTP requests. DVWA's security level was set to **Low** to establish a baseline of unmitigated vulnerabilities, consistent with the 'Low' definition in DVWA's own security settings (no security measures applied).

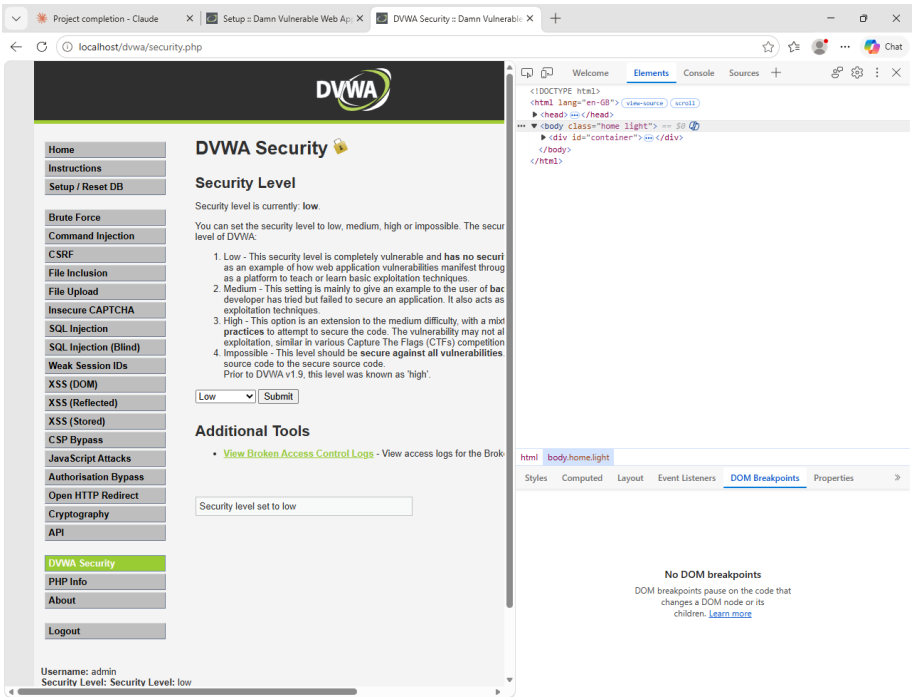


*Figure 1 — DVWA Security page confirming Security Level: Low*

**Scope:** DVWA application only, within the isolated lab environment. **Out of scope:** any production system, the underlying Windows host OS, and any third-party service.

# 3. Detailed Findings

## 3.1 SQL Injection (UNION-based)

**VULNERABILITY:** SQL Injection in DVWA 'User ID' lookup **Severity:** CRITICAL (CVSS 9.8)

**Status:** Confirmed

**Description:** The DVWA SQL Injection page (`/dvwa/vulnerabilities/sqli/`) passes the `id` parameter directly into a SQL query without sanitization or parameterization, allowing an attacker to inject arbitrary SQL. A UNION-based injection was used to extract data from the application's `users` table — including usernames and password hashes — that the query was never intended to return.

### Proof of Concept

```
Input: 1' UNION SELECT user, password FROM users-- -
```

Result: The response returned the full contents of the `users` table (usernames: admin, gordonb, 1337, pablo, smithy, and their password hashes), confirming full database read access via the injection point.

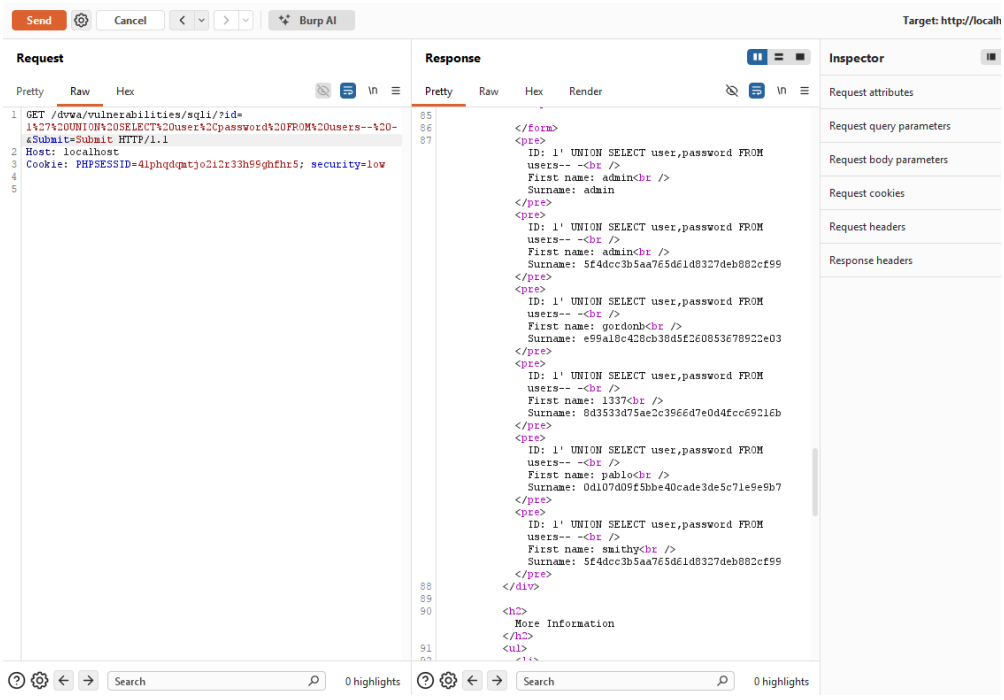


*Figure 2 — Burp Repeater: UNION-based SQL injection returning the full users table*

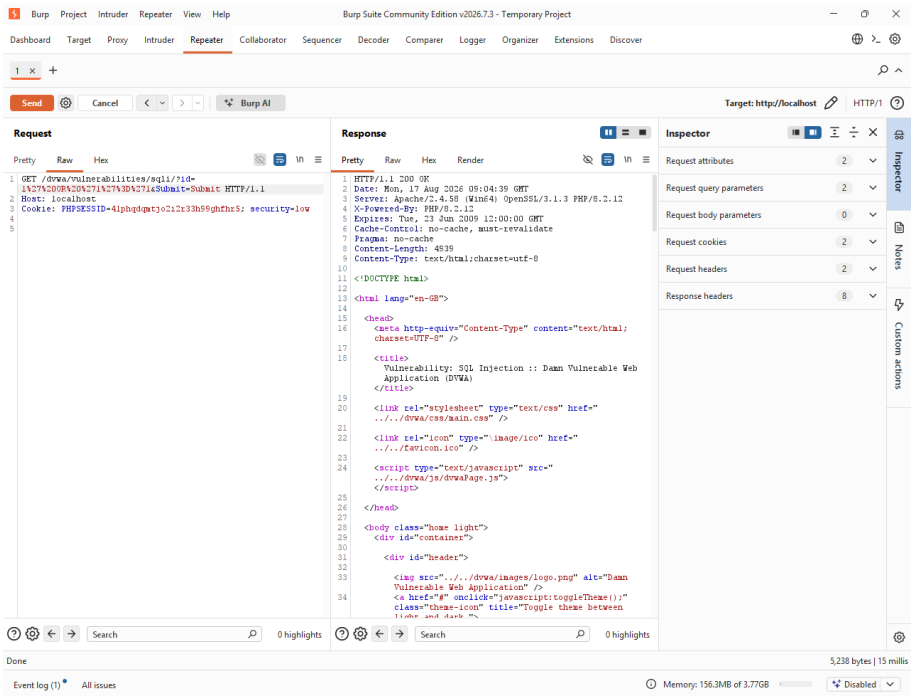


*Figure 3 — Full Burp response confirming HTTP 200 OK and page title 'SQL Injection :: DVWA'*

Earlier attempts using an unescaped single quote alone (`1' OR '1'='1` without a valid comment terminator) returned an HTTP 400 Bad Request, illustrating that malformed syntax is rejected by the web server while syntactically valid injected SQL is executed normally — confirming the query is not parameterized.

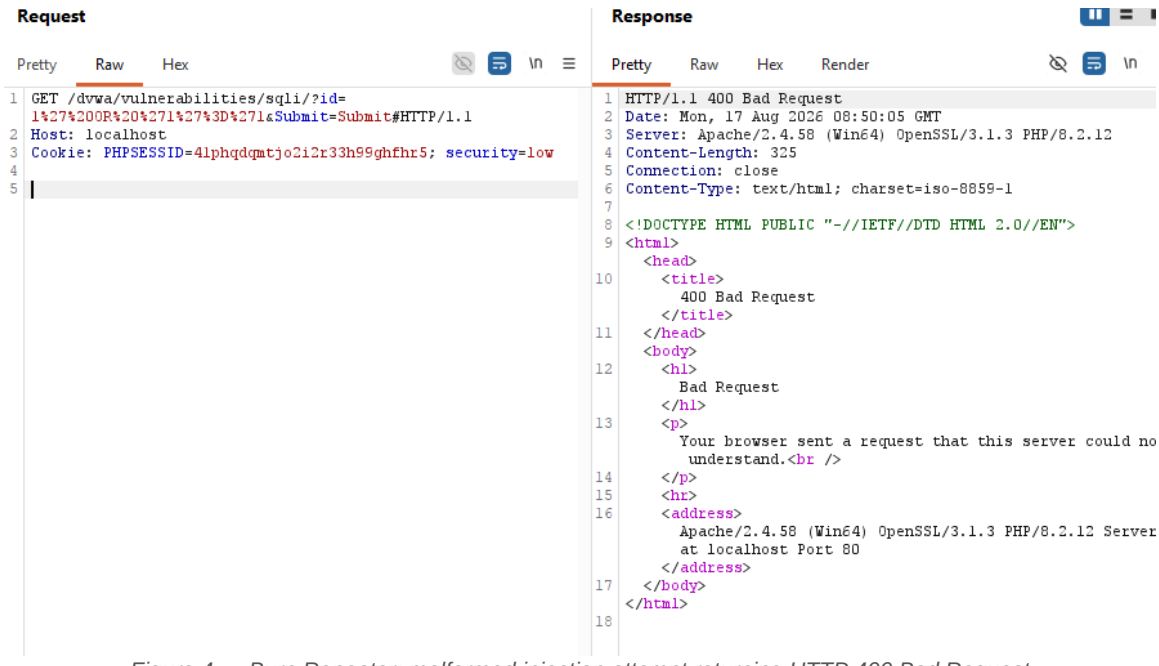


*Figure 4 — Burp Repeater: malformed injection attempt returning HTTP 400 Bad Request*

**Remediation:** (1) Use parameterized queries / prepared statements for all database access; (2) apply strict input validation and allow-listing on the `id` parameter; (3) deploy a Web Application Firewall (WAF) as defense-in-depth; (4) incorporate regular security testing (SAST/DAST) into the development lifecycle.

**Timeline:** Fix immediately (24–48 hours) | **Effort:** 4–8 developer-hours

## 3.2 Unrestricted File Upload (Remote Code Execution)

**VULNERABILITY:** Unrestricted File Upload leading to Webshell Execution

**Severity:** CRITICAL (CVSS 9.8)

**Status:** Confirmed

**Description:** The DVWA File Upload page accepts any file type without server-side validation of file extension or content (client-side/none enforced at Security Level: Low). A PHP webshell (`shell.php`) was uploaded and accepted, and stored in a publicly-accessible, web-executable directory — giving an attacker the ability to execute arbitrary PHP code, and therefore arbitrary commands, on the underlying server.

**Proof of Concept**

```
Input: uploaded a file named shell.php via the File Upload form.
```

Result: The application confirmed success with the message `../../hackable/uploads/shell.php succesfully uploaded!` — the file was written directly into the web root's uploads directory.

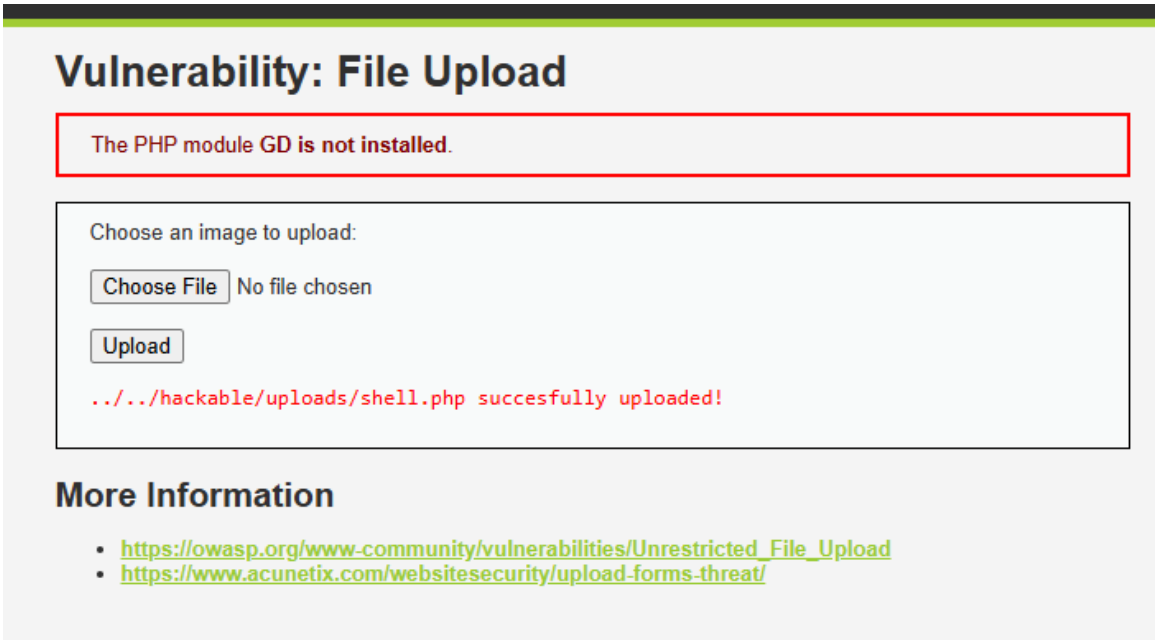


*Figure 5 — DVWA File Upload page confirming shell.php was uploaded successfully*

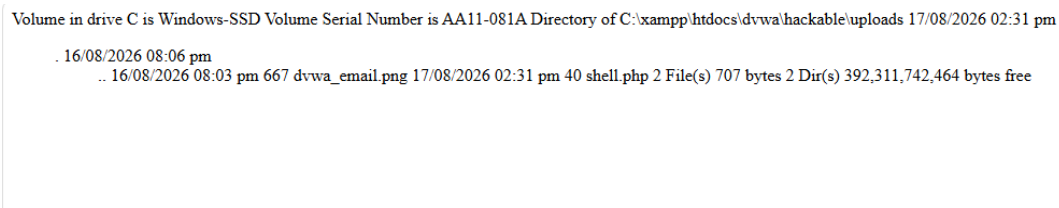


*Figure 6 — Directory listing on the host filesystem (C:\xampp\htdocs\dvwa\hackable\uploads) confirming shell.php landed alongside other uploaded files*

**Remediation:** (1) Enforce server-side allow-listing of permitted file extensions and MIME types; (2) rename uploaded files and strip executable extensions; (3) store uploads outside the web root, or serve them with execution disabled; (4) validate file content (e.g. verify true image headers) rather than trusting the filename alone.

**Timeline:** Fix immediately (24–48 hours) | **Effort:** 4–8 developer-hours

## 3.3 Stored Cross-Site Scripting (XSS)

**VULNERABILITY:** Stored XSS in DVWA Guestbook **Severity:** HIGH (CVSS 8.8)

**Status:** Confirmed

**Description:** The DVWA guestbook 'Message' field does not sanitize or encode user input before storing and rendering it back to all visitors of the page. A script payload submitted once is persisted and executes for every subsequent visitor, making this more severe than a reflected XSS since no per-victim interaction (such as clicking a crafted link) is required.

### Proof of Concept

```
Input: <script>alert('Stored XSS')</script> submitted in the guestbook Message field.
```

Result: The script executed immediately upon submission and again on every page reload, confirming the payload was stored server-side and rendered without encoding.


*Figure 7 — Stored XSS payload executing in the DVWA guestbook*

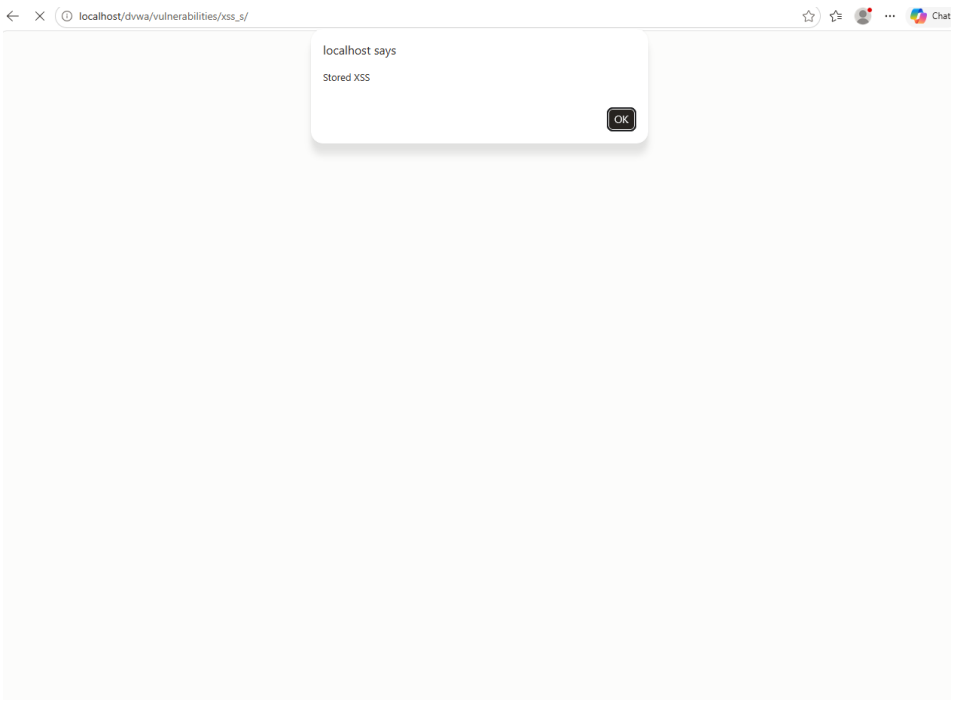


*Figure 8 — Payload re-firing on page reload, confirming persistence server-side*

**Remediation:** (1) HTML-encode all user-supplied output at render time; (2) apply a strict Content-Security-Policy to block inline script execution; (3) validate/sanitize input server-side before storage as defense-in-depth.

**Timeline:** Fix within 1–7 days | **Effort:** 2–4 developer-hours

## 3.4 Reflected Cross-Site Scripting (XSS)

**VULNERABILITY:** Reflected XSS in DVWA 'name' parameter **Severity:** MEDIUM (CVSS 6.1)

**Status:** Confirmed

**Description:** The DVWA Reflected XSS page echoes the `name` query parameter directly back into the page HTML without encoding, allowing arbitrary script execution in the context of a victim who follows a crafted link.

### Proof of Concept

```
Input: ?name=<script>alert('XSS')</script>
```

Result: The injected script executed immediately, displaying a JavaScript alert box, confirming the parameter is reflected without sanitization.

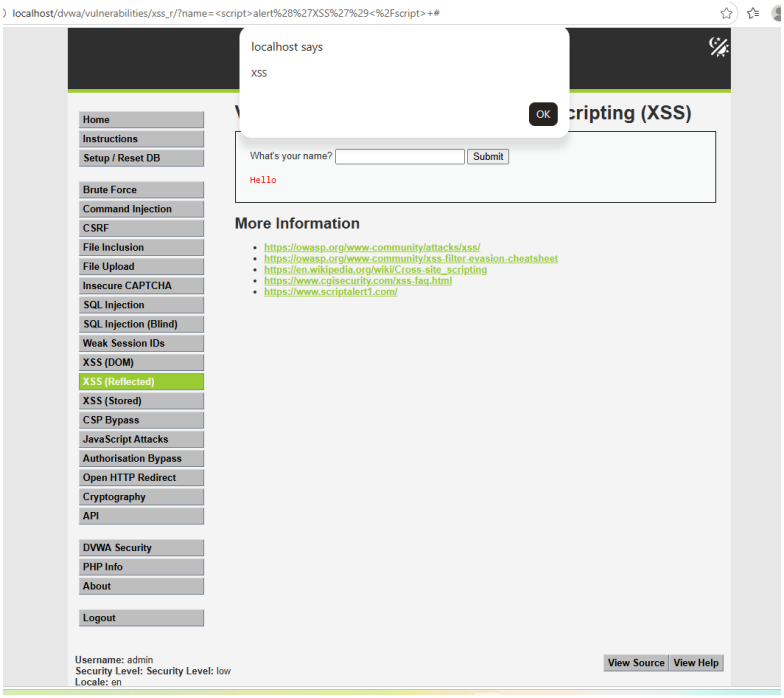


*Figure 9 — Reflected XSS payload executing via the 'name' parameter*

**Remediation:** (1) HTML-encode all reflected user input; (2) apply context-aware output encoding (HTML/JS/URL as appropriate); (3) set a Content-Security-Policy header restricting script sources.

**Timeline:** Fix within 1–30 days | **Effort:** 1–2 developer-hours

# 4. Remediation Roadmap

| Priority | Finding | Timeline |
|---|---|---|
| **Critical** | SQL Injection — implement parameterized queries | 0–24 hours |
| **Critical** | Unrestricted File Upload — enforce extension/content validation | 0–24 hours |
| **High** | Stored XSS — encode output, add CSP | 1–7 days |
| **Medium** | Reflected XSS — encode output, add CSP | 1–30 days |

# 5. Appendix — Additional Evidence

Supplementary screenshots captured during testing, including proxy/tooling configuration and environment setup.

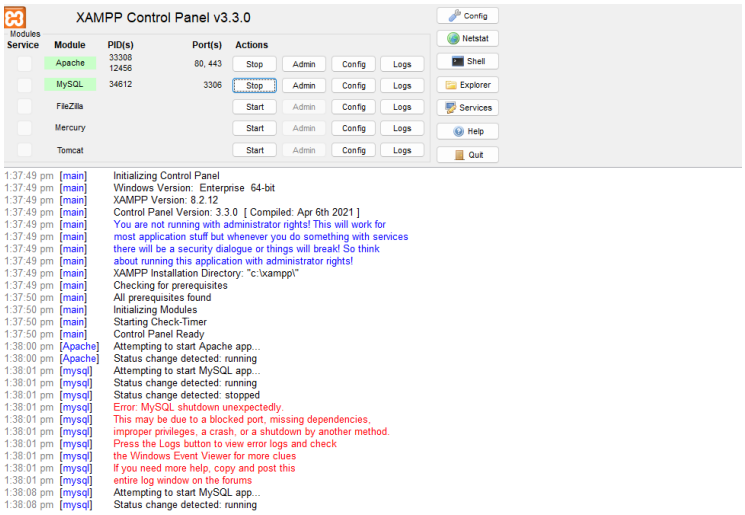


*Figure A1 — XAMPP Control Panel with Apache and MySQL running*

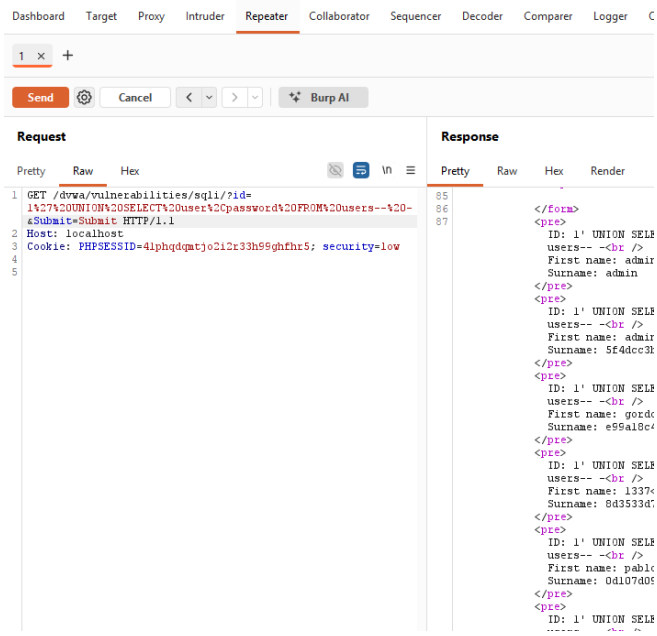


*Figure A2 — Burp Repeater: SQL injection request/response (initial capture)*

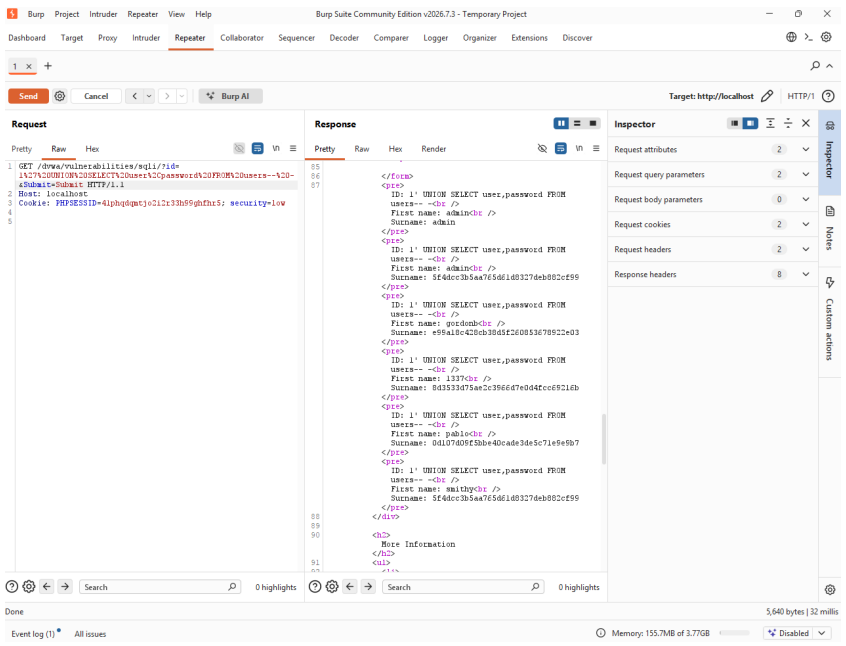


*Figure A3 — Burp Suite full window view of the UNION-based SQLi response*

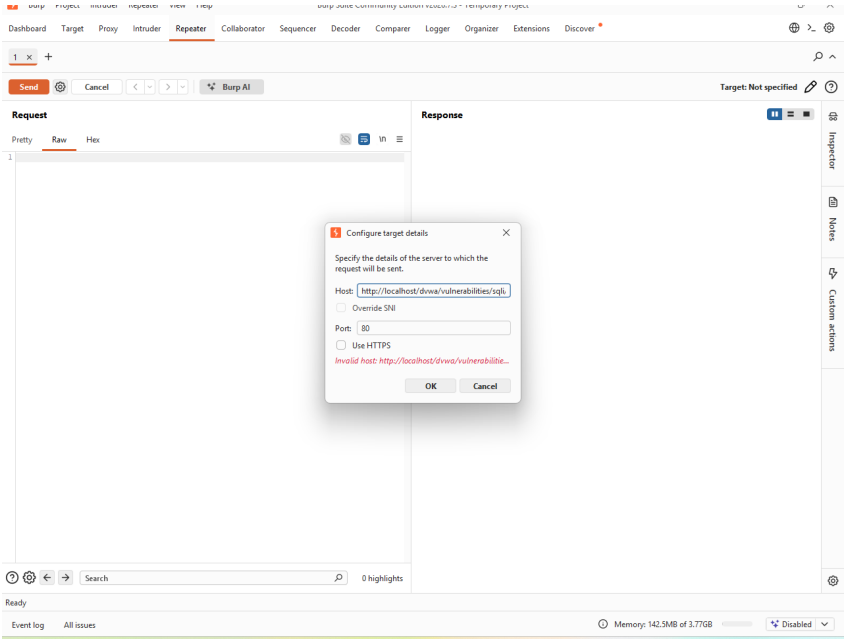


*Figure A4 — Burp Repeater target configuration dialog during environment setup*

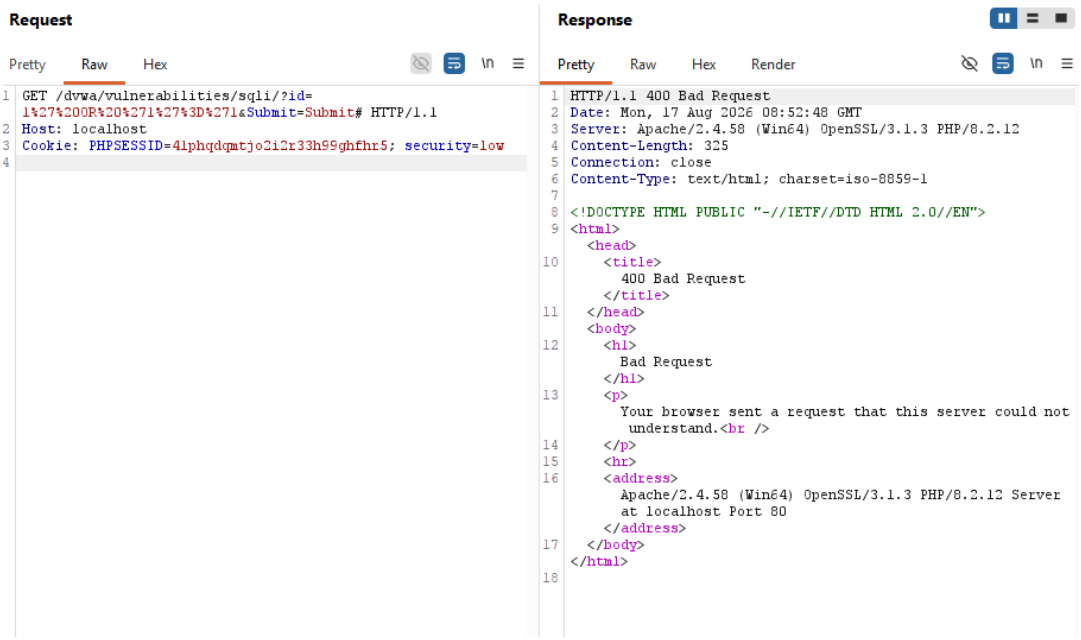


*Figure A5 — Burp Repeater: additional malformed-injection HTTP 400 response*

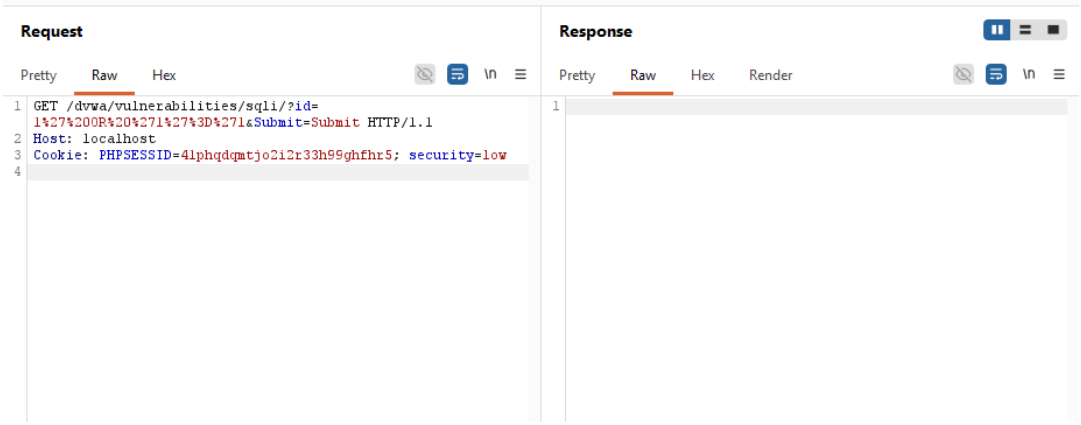


*Figure A6 — Burp Repeater: SQL injection request prepared for replay*

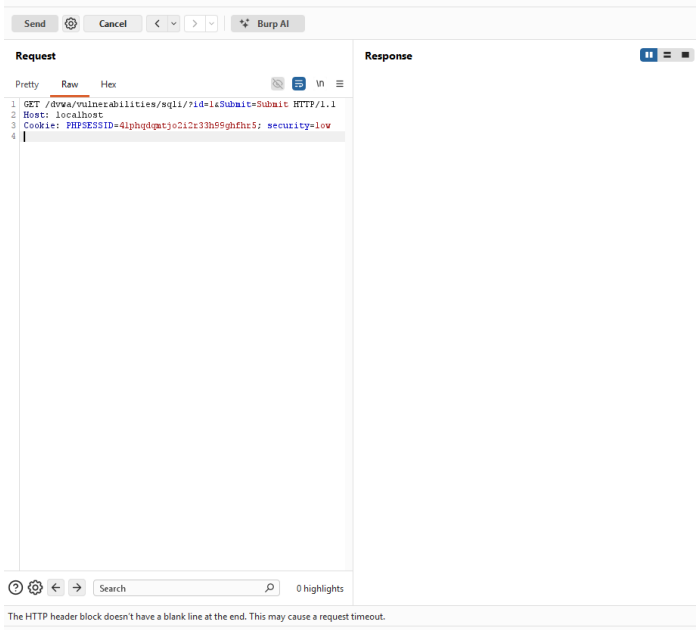


*Figure A7 — Burp Repeater request editing during payload refinement*

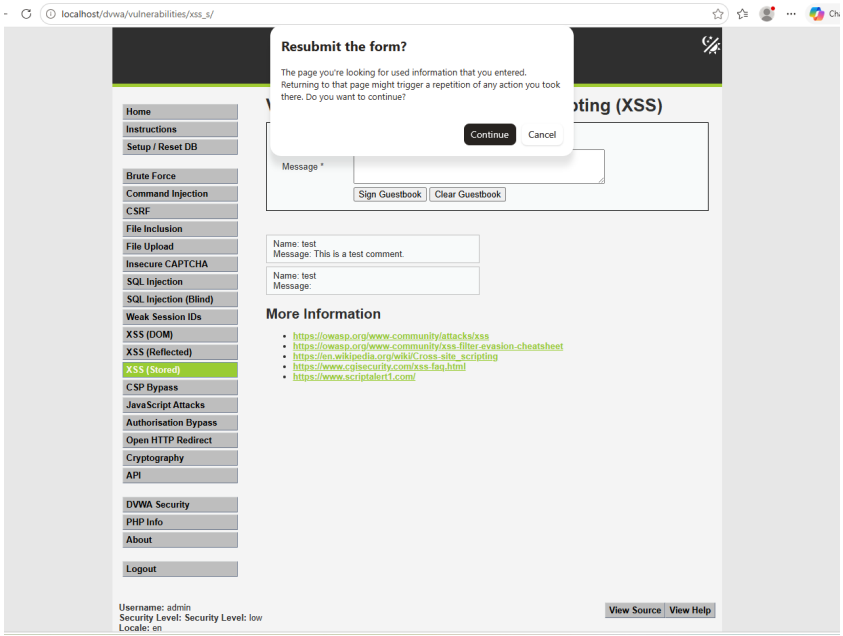


*Figure A8 — Browser 'Resubmit form' prompt confirming the stored XSS payload re-executes on reload*